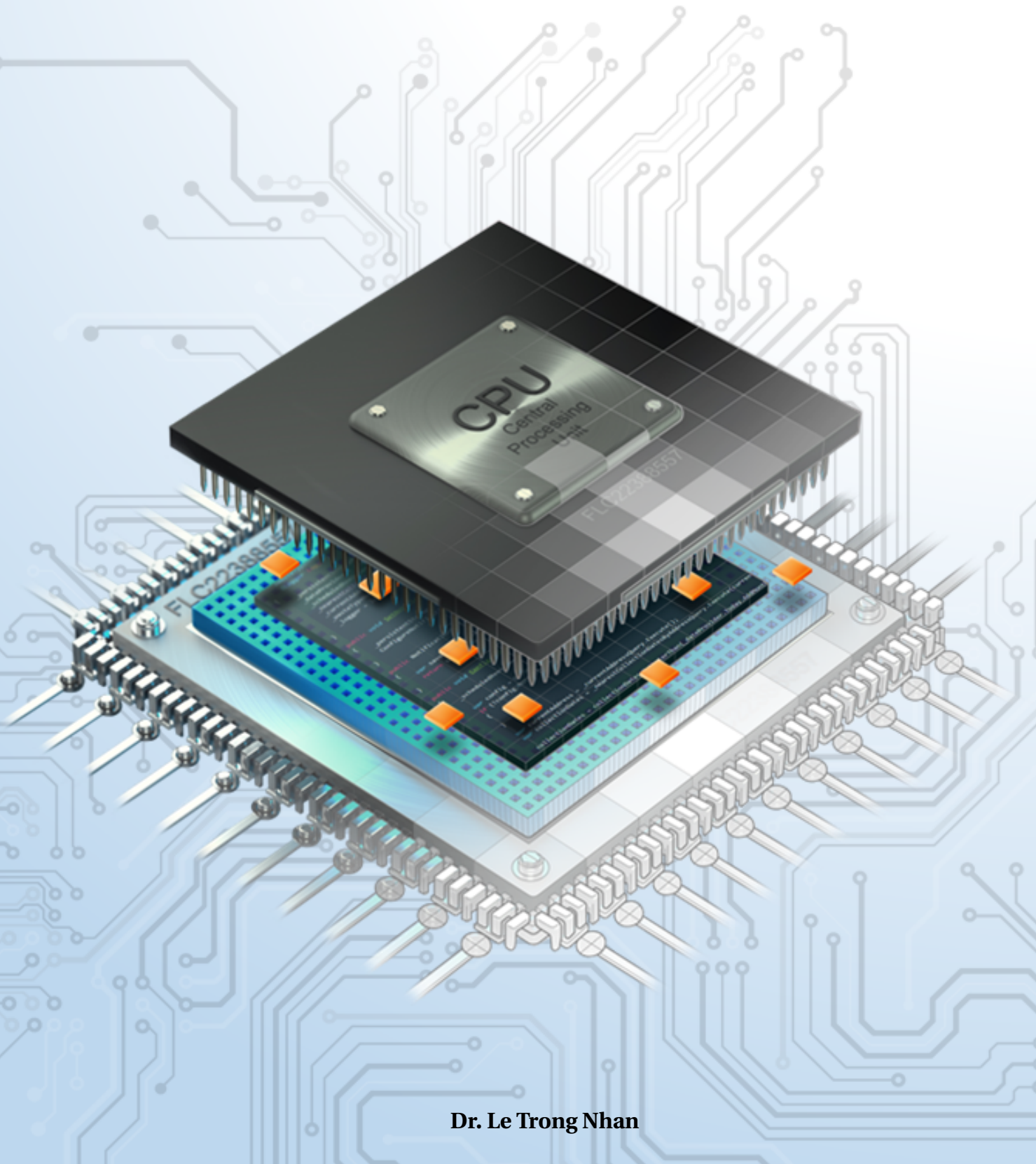




HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller



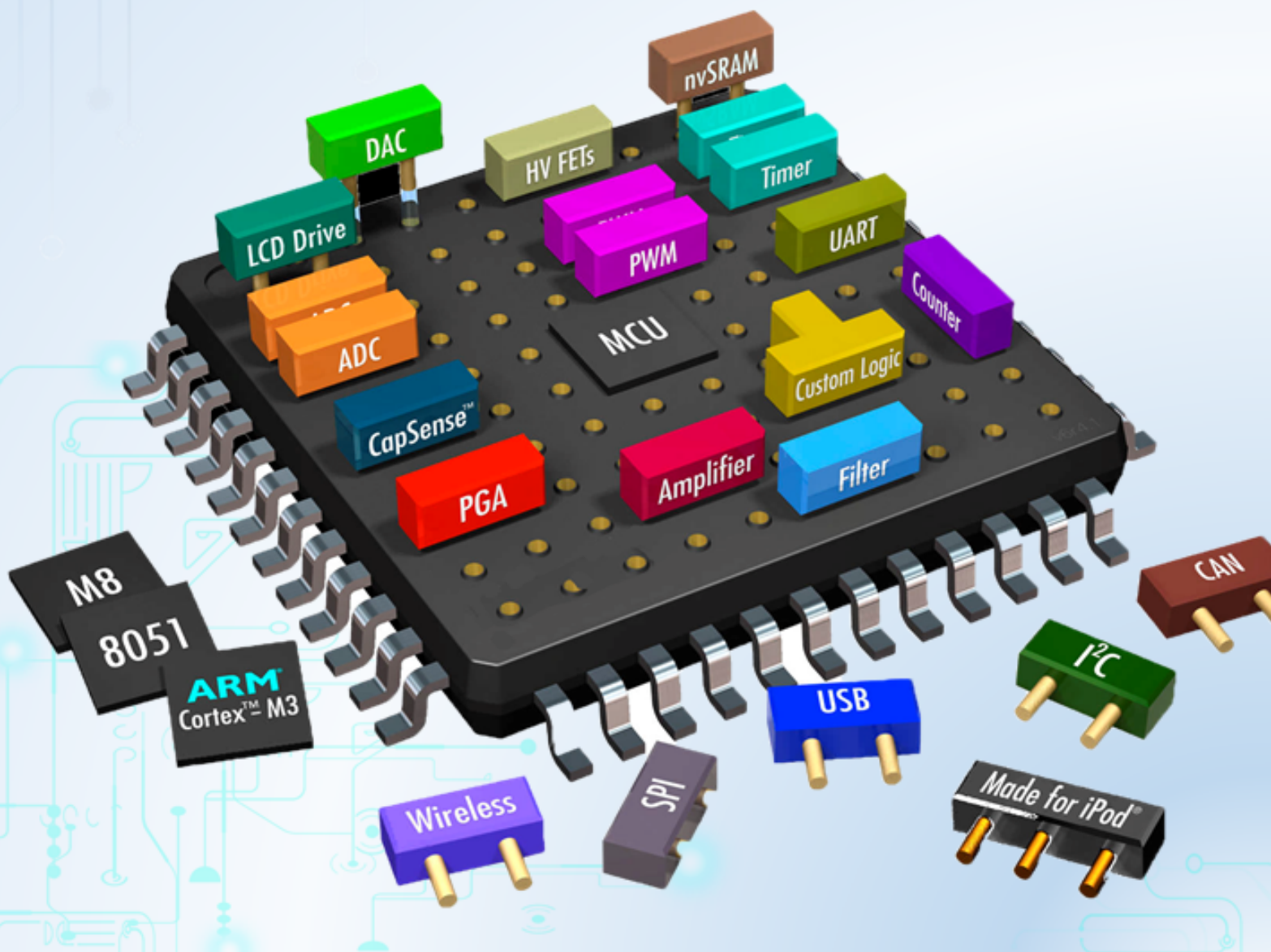
Dr. Le Trong Nhan

Mục lục

Chapter 1. Timer Interrupt and LED Scanning	7
1 Exercise and Report	8
1.1 Exercise 1	8
1.2 Exercise 2	11
1.3 Exercise 3, 8	14
1.4 Exercise 4	17
1.5 Exercise 5, 7	21
1.6 Exercise 6	25
1.7 Exercise 9	25
1.8 Exercise 10	28
2 Link Github	31

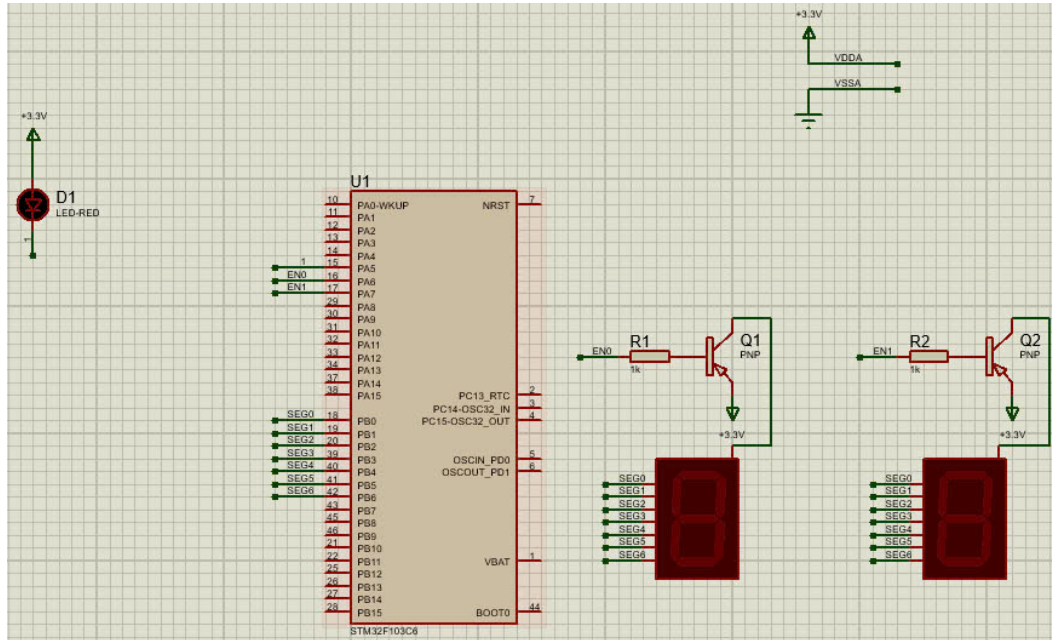
CHƯƠNG 1

Timer Interrupt and LED Scanning



1 Exercise and Report

1.1 Exercise 1



Hình 1.1: Schematic in Proteus

```
1 /* USER CODE BEGIN 2 */
2 HAL_TIM_Base_Start_IT (& htim2 ) ;
3 setTimer(0, 500);
4 /* USER CODE END 2 */
5
6 /* Infinite loop */
7 /* USER CODE BEGIN WHILE */
8 while (1)
9 {
10     if(isTimerExpired(0)==1){
11         setTimer(0, 500);
12         Ex1_run();
13     }
14     /* USER CODE END WHILE */
15
16     /* USER CODE BEGIN 3 */
17 }
```

Program 1.1: main.c

```
1 #include "software_timer1.h"
2
3 #define MAX_COUNTER 10
4 #define TIMER_TICK 10
5
```

```

6 int timer_counter[MAX_COUNTER];
7 int timer_flag[MAX_COUNTER];
8 int index_led=0;
9
10 void display7SEG(int num) {
11     const uint8_t segmentMap[10] = {
12         0b11111100,
13         0b01100000,
14         0b11011010,
15         0b11110010,
16         0b01100110,
17         0b10110110,
18         0b10111110,
19         0b11100000,
20         0b11111110,
21         0b11110110
22     };
23     HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (
segmentMap[num] & 0b10000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
24     HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, (
segmentMap[num] & 0b01000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
25     HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, (
segmentMap[num] & 0b00100000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
26     HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, (
segmentMap[num] & 0b00010000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
27     HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, (
segmentMap[num] & 0b00001000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
28     HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, (
segmentMap[num] & 0b00000100) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
29     HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, (
segmentMap[num] & 0b00000010) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
30 }
31
32 void Ex1_run(){
33     if(index_led>=2) index_led=0;
34     index_led++;
35     if(index_led<=1){
36         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
37         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, RESET);
38         display7SEG(1);
39     }
40     if(index_led>=2){

```

```

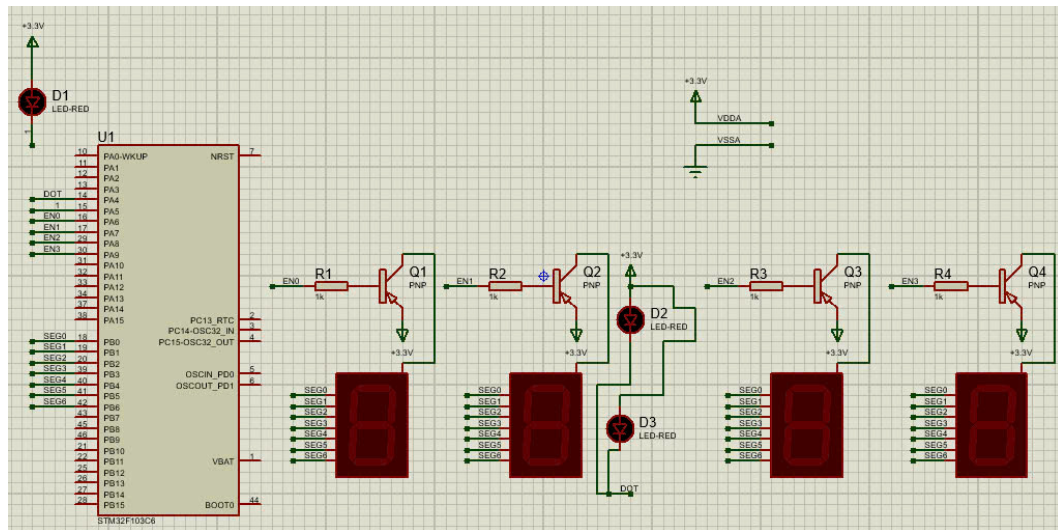
41     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
42     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, RESET);
43     display7SEG(2);
44 }
45 }
46
47 void setTimer(int index, int value){
48     timer_counter[index]=value/TIMER_TICK;
49     timer_flag[index]=0;
50 }
51
52 int isTimerExpired(int index){
53     if(timer_flag[index]==1){
54         timer_flag[index]=0;
55         return 1;
56     }
57     return 0;
58 }
59
60 void timerRun(){
61     for(int i=0; i<MAX_COUNTER; i++){
62         if(timer_counter[i]>0){
63             timer_counter[i]--;
64             if(timer_counter[i]<=0) timer_flag[i]=1;
65         }
66     }
67 }

```

Program 1.2: software_timer1.c

Short question: Tần số của quá trình quét là $\frac{1}{2 \times 0.5} = 1$ Hz.

1.2 Exercise 2



Hình 1.2: Schematic in Proteus

```

1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT (& htim2 ) ;
3  setTimer(0, 500);
4  setTimer(1, 1000);
5  /* USER CODE END 2 */
6
7  /* Infinite loop */
8  /* USER CODE BEGIN WHILE */
9  while (1)
10 {
11     if(isTimerExpired(0)==1){
12         setTimer(0, 500);
13         Ex2_run();
14     }
15     if(isTimerExpired(1)==1){
16         setTimer(1, 1000);
17         led_blinky();
18     }
19     /* USER CODE END WHILE */
20
21     /* USER CODE BEGIN 3 */
22 }

```

Program 1.3: main.c

```

1  #include "software_timer2.h"
2
3  #define MAX_COUNTER 10
4  #define TIMER_TICK 10
5
6  int timer_counter[MAX_COUNTER];

```

```

7 int timer_flag[MAX_COUNTER];
8 int index_led=0;
9 int counter=0;
10
11 void display7SEG(int num) {
12     const uint8_t segmentMap[10] = {
13         0b11111100,
14         0b01100000,
15         0b11011010,
16         0b11110010,
17         0b01100110,
18         0b10110110,
19         0b10111110,
20         0b11100000,
21         0b11111110,
22         0b11110110
23     };
24     HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (
segmentMap[num] & 0b10000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
25     HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, (
segmentMap[num] & 0b01000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
26     HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, (
segmentMap[num] & 0b00100000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
27     HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, (
segmentMap[num] & 0b00010000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
28     HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, (
segmentMap[num] & 0b00001000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
29     HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, (
segmentMap[num] & 0b00000100) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
30     HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, (
segmentMap[num] & 0b00000010) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
31 }
32
33 void Ex2_run(){
34     if(index_led>=4) index_led=0;
35     index_led++;
36     if(index_led<=1){
37         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
38         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
39         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
40         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, RESET);
41         display7SEG(1);

```

```

42 }
43 if(index_led>=2&&index_led<3){
44     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
45     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
46     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
47     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, RESET);
48     display7SEG(2);
49 }
50 if(index_led>=3&&index_led<4){
51     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
52     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
53     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
54     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, RESET);
55     display7SEG(3);
56 }
57 if(index_led>=4){
58     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
59     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
60     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
61     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, RESET);
62     display7SEG(0);
63 }
64 }
65
66 void led_blinky(){
67     if(counter>=2) counter=0;
68     counter++;
69     if(counter<=1) HAL_GPIO_WritePin(DOT_GPIO_Port,DOT_Pin ,
70     RESET);
71     else HAL_GPIO_WritePin(DOT_GPIO_Port,DOT_Pin , SET);
72 }
73
74 void setTimer(int index, int value){
75     timer_counter[index]=value/TIMER_TICK;
76     timer_flag[index]=0;
77 }
78
79 int isTimerExpired(int index){
80     if(timer_flag[index]==1){
81         timer_flag[index]=0;
82         return 1;
83     }
84     return 0;
85 }
86
87 void timerRun(){
88     for(int i=0;i<MAX_COUNTER;i++){
89         if(timer_counter[i]>0){
90             timer_counter[i]--;

```

```

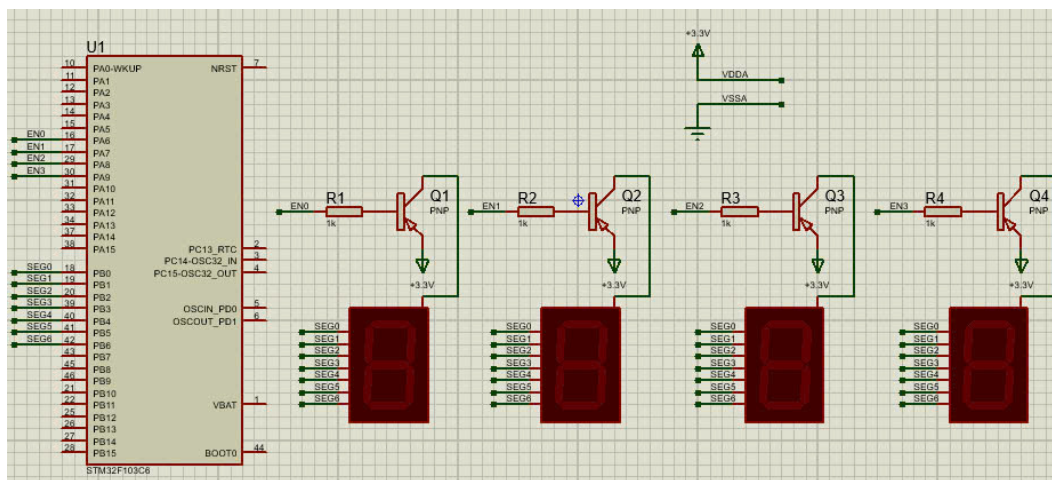
90     if(timer_counter[i]<=0) timer_flag[i]=1;
91 }
92 }
93 }

```

Program 1.4: software_timer2.c

Short question: Tần số của quá trình quét là $\frac{1}{4 \times 0.5} = 0.5 \text{ Hz}$.

1.3 Exercise 3, 8



Hình 1.3: Schematic in Proteus

```

1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT (& htim2 ) ;
3  setTimer(0, 500);
4  /* USER CODE END 2 */
5
6  /* Infinite loop */
7  /* USER CODE BEGIN WHILE */
8  while (1)
9  {
10     if(isTimerExpired(0)==1){
11         setTimer(0, 500);
12         Ex3_run();
13     }
14     /* USER CODE END WHILE */
15
16     /* USER CODE BEGIN 3 */
17 }

```

Program 1.5: main.c

```

1 #include "software_timer3.h"
2
3 #define MAX_COUNTER 10

```

```

4 #define TIMER_TICK 10
5
6 int timer_counter[MAX_COUNTER];
7 int timer_flag[MAX_COUNTER];
8 const int MAX_LED = 4;
9 int index_led = 0;
10 int led_buffer [4] = {1 , 2 , 3 , 4};
11
12 void display7SEG(int num) {
13     const uint8_t segmentMap[10] = {
14         0b11111100,
15         0b01100000,
16         0b11011010,
17         0b11110010,
18         0b01100110,
19         0b10110110,
20         0b10111110,
21         0b11100000,
22         0b11111110,
23         0b11110110
24     };
25     HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (
segmentMap[num] & 0b10000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
26     HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, (
segmentMap[num] & 0b01000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
27     HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, (
segmentMap[num] & 0b00100000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
28     HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, (
segmentMap[num] & 0b00010000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
29     HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, (
segmentMap[num] & 0b00001000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
30     HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, (
segmentMap[num] & 0b00000100) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
31     HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, (
segmentMap[num] & 0b00000010) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
32 }
33
34 void update7SEG ( int index ) {
35     switch ( index ) {
36     case 0:
37         // Display the first 7 SEG with led_buffer [0]
38         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);

```

```

39     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
40     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
41     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, RESET);
42     display7SEG(led_buffer[index]);
43     break ;
44     case 1:
45     // Display the second 7 SEG with led_buffer [1]
46     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
47     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
48     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
49     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, RESET);
50     display7SEG(led_buffer[index]);
51     break ;
52     case 2:
53     // Display the third 7 SEG with led_buffer [2]
54     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
55     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
56     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
57     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, RESET);
58     display7SEG(led_buffer[index]);
59     break ;
60     case 3:
61     // Display the forth 7 SEG with led_buffer [3]
62     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
63     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
64     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
65     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, RESET);
66     display7SEG(led_buffer[index]);
67     break ;
68     default :
69     HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
70     HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
71     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
72     HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
73     break ;
74 }
75 }
76
77 void Ex3_run(){
78     if(index_led>=MAX_LED) index_led=0;
79     update7SEG(index_led++);
80 }
81
82 void setTimer(int index, int value){
83     timer_counter[index]=value/TIMER_TICK;
84     timer_flag[index]=0;
85 }
86
87 int isTimerExpired(int index){

```

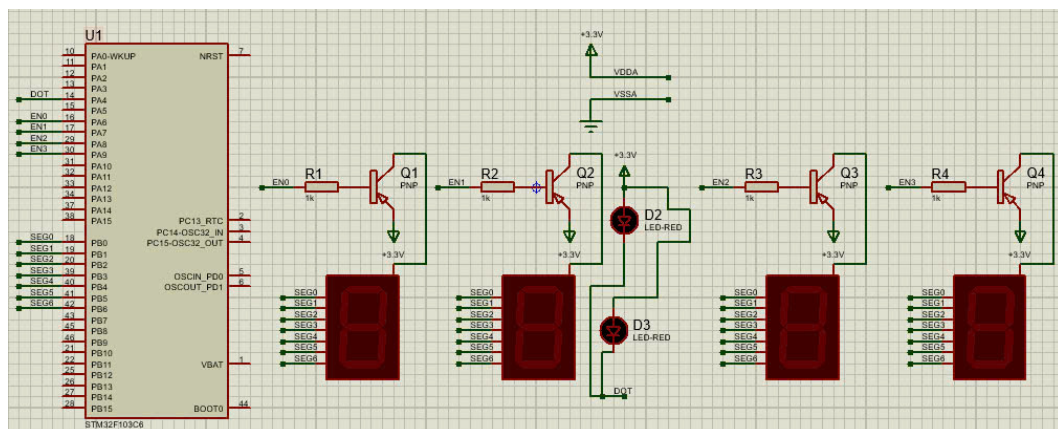
```

88     if(timer_flag[index]==1){
89         timer_flag[index]=0;
90         return 1;
91     }
92     return 0;
93 }
94
95 void timerRun(){
96     for(int i=0;i<MAX_COUNTER;i++){
97         if(timer_counter[i]>0){
98             timer_counter[i]--;
99             if(timer_counter[i]<=0) timer_flag[i]=1;
100         }
101     }
102 }

```

Program 1.6: software_timer3.c

1.4 Exercise 4



Hình 1.4: Schematic in Proteus

```

1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT (& htim2 ) ;
3  setTimer(0, 250);
4  setTimer(1, 1000);
5  /* USER CODE END 2 */
6
7  /* Infinite loop */
8  /* USER CODE BEGIN WHILE */
9  while (1)
10 {
11     if(isTimerExpired(0)==1){
12         setTimer(0, 250);
13         Ex4_run();
14     }

```

```

15     if(isTimerExpired(1)==1){
16         setTimer(1, 1000);
17         led_blinky();
18     }
19     /* USER CODE END WHILE */
20
21     /* USER CODE BEGIN 3 */
22 }

```

Program 1.7: main.c

```

1  #include "software_timer4.h"
2
3  #define MAX_COUNTER 10
4  #define TIMER_TICK 10
5
6  int timer_counter[MAX_COUNTER];
7  int timer_flag[MAX_COUNTER];
8  int counter=0;
9  const int MAX_LED = 4;
10 int index_led = 0;
11 int led_buffer [4] = {1 , 2 , 3 , 4};
12
13 void display7SEG(int num) {
14     const uint8_t segmentMap[10] = {
15         0b11111100,
16         0b01100000,
17         0b11011010,
18         0b11110010,
19         0b01100110,
20         0b10110110,
21         0b10111110,
22         0b11100000,
23         0b11111110,
24         0b11110110
25     };
26     HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (
segmentMap[num] & 0b10000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
27     HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, (
segmentMap[num] & 0b01000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
28     HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, (
segmentMap[num] & 0b00100000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
29     HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, (
segmentMap[num] & 0b00010000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
30     HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, (
segmentMap[num] & 0b00001000) ? GPIO_PIN_RESET :

```

```

GPIO_PIN_SET);
31     HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, (
segmentMap[num] & 0b00000100) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
32     HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, (
segmentMap[num] & 0b00000010) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
33 }
34
35 void update7SEG ( int index ) {
36     switch ( index ) {
37     case 0:
38         // Display the first 7 SEG with led_buffer [0]
39         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
40         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
41         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
42         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, RESET);
43         display7SEG(led_buffer[index]);
44         break ;
45     case 1:
46         // Display the second 7 SEG with led_buffer [1]
47         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
48         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
49         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
50         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, RESET);
51         display7SEG(led_buffer[index]);
52         break ;
53     case 2:
54         // Display the third 7 SEG with led_buffer [2]
55         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
56         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
57         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
58         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, RESET);
59         display7SEG(led_buffer[index]);
60         break ;
61     case 3:
62         // Display the forth 7 SEG with led_buffer [3]
63         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
64         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
65         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
66         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, RESET);
67         display7SEG(led_buffer[index]);
68         break ;
69     default :
70         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
71         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
72         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
73         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
74         break ;

```

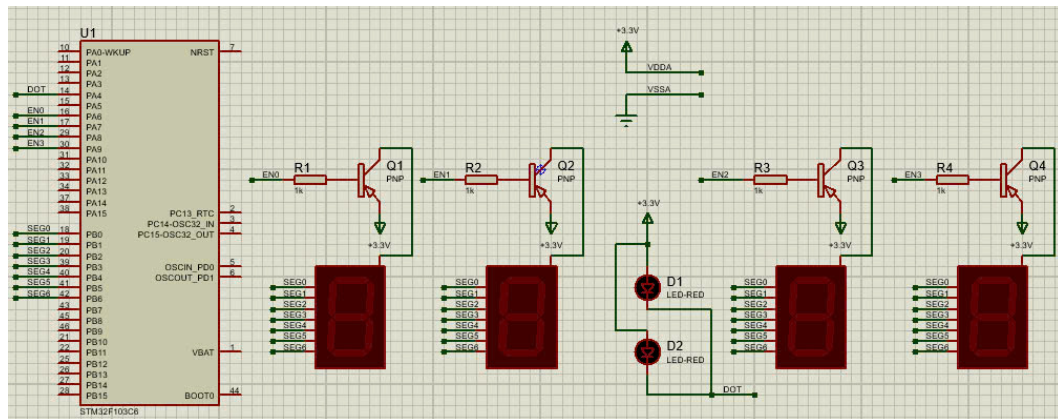
```

75     }
76 }
77
78 void led_blinky(){
79     if(counter>=2) counter=0;
80     counter++;
81     if(counter<=1) HAL_GPIO_WritePin(DOT_GPIO_Port,DOT_Pin ,
82         RESET);
83     else HAL_GPIO_WritePin(DOT_GPIO_Port,DOT_Pin , SET);
84 }
85
86 void Ex4_run(){
87     if(index_led>=MAX_LED) index_led=0;
88     update7SEG(index_led++);
89 }
90
91 void setTimer(int index, int value){
92     timer_counter[index]=value/TIMER_TICK;
93     timer_flag[index]=0;
94 }
95
96 int isTimerExpired(int index){
97     if(timer_flag[index]==1){
98         timer_flag[index]=0;
99         return 1;
100     }
101     return 0;
102 }
103
104 void timerRun(){
105     for(int i=0;i<MAX_COUNTER;i++){
106         if(timer_counter[i]>0){
107             timer_counter[i]--;
108             if(timer_counter[i]<=0) timer_flag[i]=1;
109         }
110     }

```

Program 1.8: software_timer4.c

1.5 Exercise 5, 7



Hình 1.5: Schematic in Proteus

```

1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT (& htim2 ) ;
3      setTimer(0, 250);
4      setTimer(1, 1000);
5      setTimer(2, 1000);
6  /* USER CODE END 2 */
7
8  /* Infinite loop */
9  /* USER CODE BEGIN WHILE */
10 while (1)
11 {
12     if(isTimerExpired(0)==1){
13         setTimer(0, 250);
14         scan_7LED();
15     }
16     if(isTimerExpired(1)==1){
17         setTimer(1, 1000);
18         Ex5_run();
19     }
20     if(isTimerExpired(2)==1){
21         setTimer(2, 1000);
22         led_blinky();
23     }
24
25     /* USER CODE END WHILE */
26
27     /* USER CODE BEGIN 3 */
28 }

```

Program 1.9: main.c

```

1  #include "software_timer5.h"
2  #define MAX_COUNTER 10
3  #define TIMER_TICK 10

```

```

4
5 int timer_counter[MAX_COUNTER];
6 int timer_flag[MAX_COUNTER];
7
8 int hour = 15 , minute = 8 , second = 50;
9 int counter=0;
10 const int MAX_LED = 4;
11 int index_led = 0;
12 int led_buffer [4] = {1 , 5 , 0 , 8};
13
14 void updateClockBuffer (){
15     led_buffer[0]=hour/10;
16     led_buffer[1]=hour%10;
17     led_buffer[2]=minute/10;
18     led_buffer[3]=minute%10;
19 }
20 void Ex5_run(){
21     second ++;
22     if ( second >= 60) {
23         second = 0;
24         minute ++;
25     }
26     if( minute >= 60) {
27         minute = 0;
28         hour ++;
29     }
30     if( hour >=24) {
31         hour = 0;
32     }
33     updateClockBuffer();
34 }
35
36 void display7SEG(int num) {
37     const uint8_t segmentMap[10] = {
38         0b11111100, // 0
39         0b01100000, // 1
40         0b11011010, // 2
41         0b11110010, // 3
42         0b01100110, // 4
43         0b10110110, // 5
44         0b10111110, // 6
45         0b11100000, // 7
46         0b11111110, // 8
47         0b11110110 // 9
48     };
49     HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, (
50         segmentMap[num] & 0b10000000) ? GPIO_PIN_RESET :
51         GPIO_PIN_SET);
52     HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, (

```

```

segmentMap[num] & 0b01000000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
51     HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, (
segmentMap[num] & 0b00100000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
52     HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, (
segmentMap[num] & 0b00010000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
53     HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, (
segmentMap[num] & 0b00001000) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
54     HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, (
segmentMap[num] & 0b00000100) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
55     HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, (
segmentMap[num] & 0b00000010) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
56 }
57 void update7SEG ( int index ) {
58     switch ( index ) {
59     case 0:
60         // Display the first 7 SEG with led_buffer [0]
61         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
62         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
63         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
64         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, RESET);
65         display7SEG(led_buffer[index]);
66         break ;
67     case 1:
68         // Display the second 7 SEG with led_buffer [1]
69         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
70         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
71         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
72         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, RESET);
73         display7SEG(led_buffer[index]);
74         break ;
75     case 2:
76         // Display the third 7 SEG with led_buffer [2]
77         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
78         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
79         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
80         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, RESET);
81         display7SEG(led_buffer[index]);
82         break ;
83     case 3:
84         // Display the forth 7 SEG with led_buffer [3]
85         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
86         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
87         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);

```

```

88     HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, RESET);
89     display7SEG(led_buffer[index]);
90     break ;
91     default :
92         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, SET);
93         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, SET);
94         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, SET);
95         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, SET);
96     break ;
97 }
98 }
99 void led_blinky(){
100     if(counter>=2) counter=0;
101     counter++;
102     if(counter<=1) HAL_GPIO_WritePin(DOT_GPIO_Port,DOT_Pin ,
103         RESET);
104     else HAL_GPIO_WritePin(DOT_GPIO_Port,DOT_Pin , SET);
105 }
106 void scan_7LED(){
107     if(index_led>=MAX_LED) {
108         index_led=0;
109     }
110     update7SEG(index_led++);
111 }
112
113 void setTimer(int index, int value){
114     timer_counter[index]=value/TIMER_TICK;
115     timer_flag[index]=0;
116 }
117
118 int isTimerExpired(int index){
119     if(timer_flag[index]==1){
120         timer_flag[index]=0;
121         return 1;
122     }
123     return 0;
124 }
125
126 void timerRun(){
127     for(int i=0;i<MAX_COUNTER;i++){
128         if(timer_counter[i]>0){
129             timer_counter[i]--;
130             if(timer_counter[i]<=0) timer_flag[i]=1;
131         }
132     }
133 }

```

Program 1.10: software_timer5.c

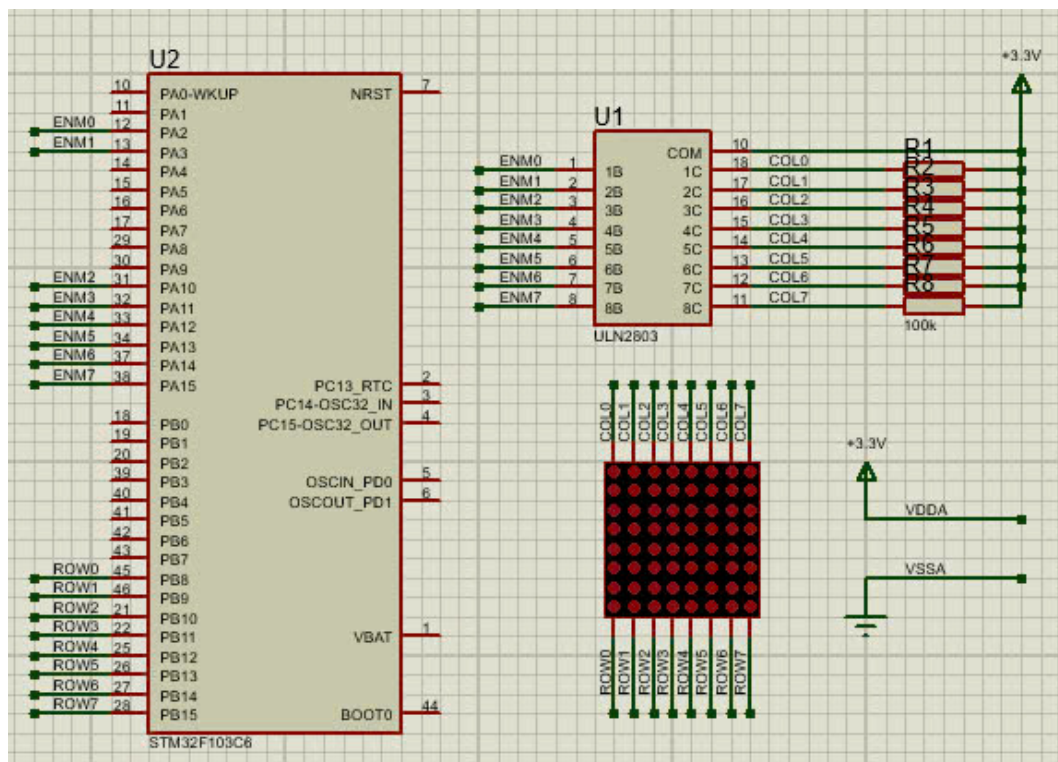
1.6 Exercise 6

Nếu dòng 1 của đoạn mã trên bị miss thì chỉ còn 99 lần gọi hàm ngắt mỗi 10 ms do lần đầu chạy quá nhanh nó xảy ra ngắt liền, sai số về thời gian lúc này chỉ là $\frac{1}{100} = 1\%$.

Nếu dòng 1 của đoạn mã trên bị thay đổi thành `setTimer0(1)`, thì vòng lặp `while` không gọi hàm ngắt mỗi 10 ms được vì sai số về thời gian lúc này lên đến là $\frac{1}{0.1} = 1000\%$.

Nếu dòng 1 của đoạn mã trên bị thay đổi thành `setTimer0(10)`, thì vòng lặp `while` không gọi hàm ngắt mỗi 10 ms được vì sai số về thời gian lúc này lên đến là $\frac{1}{1} = 100\%$ dẫn đến đèn led không chớp tắt được.

1.7 Exercise 9



Hình 1.6: LED matrix is added to the simulation

```

1 /* USER CODE BEGIN 2 */
2 HAL_TIM_Base_Start_IT (& htim2 ) ;
3 setTimer(0, 10);
4 /* USER CODE END 2 */
5
6 /* Infinite loop */
7 /* USER CODE BEGIN WHILE */
8 while (1)
9 {

```

```

10     if(isTimerExpired(0)==1){
11         setTimer(0, 10);
12         Ex9_run();
13     }
14     /* USER CODE END WHILE */
15
16     /* USER CODE BEGIN 3 */
17 }

```

Program 1.11: main.c

```

1 #include "software_timer9.h"
2
3 #define MAX_COUNTER 10
4 #define TIMER_TICK 10
5
6 int timer_counter[MAX_COUNTER];
7 int timer_flag[MAX_COUNTER];
8 const int MAX_LED_MATRIX = 8;
9 int index_led_matrix = 0;
10 uint8_t matrix_buffer [8] = {0xFF , 0xC0 , 0x80 , 0x33 , 0
    x33 , 0x80 , 0xC0 , 0xFF };
11 uint16_t segmentPins [8] = {ROW0_Pin,ROW1_Pin,ROW2_Pin,
    ROW3_Pin,ROW4_Pin,ROW5_Pin,ROW6_Pin,ROW7_Pin};
12
13 void displayMatrix(uint8_t num){
14     for(int i=0;i<MAX_LED_MATRIX;i++){
15         HAL_GPIO_WritePin(GPIOB, segmentPins [i], (num&(0x80>>i
    ))?SET:RESET);
16     }
17 }
18
19 void Ex9_run(){
20     if(index_led_matrix>=8) index_led_matrix=0;
21     updateLEDMatrix(index_led_matrix++);
22 }
23
24 void updateLEDMatrix (int index ) {
25     HAL_GPIO_WritePin(GPIOA, LED_Pins, SET);
26     switch ( index ) {
27     case 0:
28         HAL_GPIO_WritePin(GPIOA, ENM0_Pin, RESET);
29         displayMatrix(matrix_buffer[index]);
30         break ;
31     case 1:
32         HAL_GPIO_WritePin(GPIOA, ENM1_Pin, RESET);
33         displayMatrix(matrix_buffer[index]);
34         break ;
35     case 2:
36         HAL_GPIO_WritePin(GPIOA, ENM2_Pin, RESET);

```

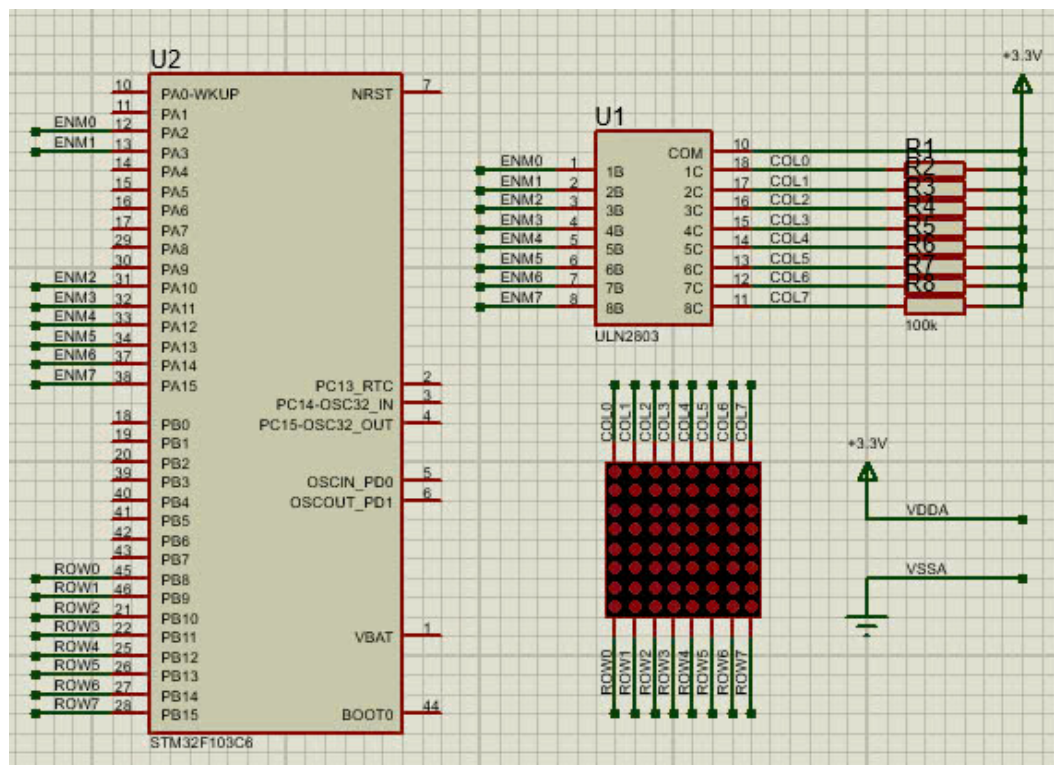
```

37     displayMatrix(matrix_buffer[index]);
38     break ;
39     case 3:
40         HAL_GPIO_WritePin(GPIOA, ENM3_Pin, RESET);
41         displayMatrix(matrix_buffer[index]);
42     break ;
43     case 4:
44         HAL_GPIO_WritePin(GPIOA, ENM4_Pin, RESET);
45         displayMatrix(matrix_buffer[index]);
46     break ;
47     case 5:
48         HAL_GPIO_WritePin(GPIOA, ENM5_Pin, RESET);
49         displayMatrix(matrix_buffer[index]);
50     break ;
51     case 6:
52         HAL_GPIO_WritePin(GPIOA, ENM6_Pin, RESET);
53         displayMatrix(matrix_buffer[index]);
54     break ;
55     case 7:
56         HAL_GPIO_WritePin(GPIOA, ENM7_Pin, RESET);
57         displayMatrix(matrix_buffer[index]);
58     break ;
59     default :
60         HAL_GPIO_WritePin(GPIOA, LED_Pins , SET);
61     break ;
62 }
63 }
64
65 void setTimer(int index, int value){
66     timer_counter[index]=value/TIMER_TICK;
67     timer_flag[index]=0;
68 }
69 int isTimerExpired(int index){
70     if(timer_flag[index]==1){
71         timer_flag[index]=0;
72         return 1;
73     }
74     return 0;
75 }
76 void timerRun(){
77     for(int i=0;i<MAX_COUNTER;i++){
78         if(timer_counter[i]>0){
79             timer_counter[i]--;
80             if(timer_counter[i]<=0) timer_flag[i]=1;
81         }
82     }
83 }

```

Program 1.12: software_timer9.c

1.8 Exercise 10



Hình 1.7: LED matrix is added to the simulation

```

1  /* USER CODE BEGIN 2 */
2  HAL_TIM_Base_Start_IT (& htim2 ) ;
3  setTimer(0, 10);
4  setTimer(1, 80);
5  /* USER CODE END 2 */
6
7  /* Infinite loop */
8  /* USER CODE BEGIN WHILE */
9  while (1)
10 {
11     if(isTimerExpired(0)==1){
12         setTimer(0, 10);
13         Ex10_run();
14     }
15     if(isTimerExpired(1)==1){
16         setTimer(1, 80);
17         // shiftColLeft();
18         // shiftColRight();
19         // shiftRowUp();
20         // shiftRowDown();
21     }
22     /* USER CODE END WHILE */
23
24     /* USER CODE BEGIN 3 */

```

25 }

Program 1.13: main.c

```
1 #include "software_timer10.h"
2
3 #define MAX_COUNTER 10
4 #define TIMER_TICK 10
5
6 int timer_counter[MAX_COUNTER];
7 int timer_flag[MAX_COUNTER];
8 const int MAX_LED_MATRIX = 8;
9 int index_led_matrix = 0;
10 uint8_t matrix_buffer [8] = {0xFF , 0xC0 , 0x80 , 0x33 , 0
    x33 , 0x80 , 0xC0 , 0xFF };
11 uint16_t segmentPins[8] = {ROW0_Pin,ROW1_Pin,ROW2_Pin,
    ROW3_Pin,ROW4_Pin,ROW5_Pin,ROW6_Pin,ROW7_Pin};
12
13 void displayMatrix(uint8_t num){
14     for(int i=0;i<MAX_LED_MATRIX;i++){
15         HAL_GPIO_WritePin(GPIOB, segmentPins[i], (num&(0x80>>i
            ))?SET:RESET);
16     }
17 }
18
19 void shiftColLeft(){
20     for(int i=0;i<MAX_LED_MATRIX-1;i++){
21         matrix_buffer[i]=matrix_buffer[i+1];
22     }
23     matrix_buffer[7]=matrix_buffer[0];
24 }
25
26 void shiftColRight(){
27     for(int i=MAX_LED_MATRIX-1;i>0;i--){
28         matrix_buffer[i]=matrix_buffer[i-1];
29     }
30     matrix_buffer[0]=matrix_buffer[7];
31 }
32
33 void shiftRowUp(){
34     for (int i = 0; i < MAX_LED_MATRIX; i++) {
35         uint8_t temp = (matrix_buffer[i] & 0x80) ? 1 :
            0;
36         matrix_buffer[i] <= 1;
37         matrix_buffer[i] |= temp;
38     }
39 }
40
41 void shiftRowDown(){
42     for (int i = 0; i < MAX_LED_MATRIX; i++) {
```

```

43     uint8_t temp = (matrix_buffer[i] & 0x01) ? 0x80 : 0;
44         matrix_buffer[i] >>= 1;
45         matrix_buffer[i] |= temp;
46     }
47 }
48
49 void Ex10_run(){
50     if(index_led_matrix>=8) index_led_matrix=0;
51     updateLEDMatrix(index_led_matrix++);
52 }
53
54 void updateLEDMatrix (int index ) {
55     HAL_GPIO_WritePin(GPIOA, LED_Pins, SET);
56     switch ( index ) {
57         case 0:
58             HAL_GPIO_WritePin(GPIOA, ENM0_Pin, RESET);
59             displayMatrix(matrix_buffer[index]);
60             break ;
61         case 1:
62             HAL_GPIO_WritePin(GPIOA, ENM1_Pin, RESET);
63             displayMatrix(matrix_buffer[index]);
64             break ;
65         case 2:
66             HAL_GPIO_WritePin(GPIOA, ENM2_Pin, RESET);
67             displayMatrix(matrix_buffer[index]);
68             break ;
69         case 3:
70             HAL_GPIO_WritePin(GPIOA, ENM3_Pin, RESET);
71             displayMatrix(matrix_buffer[index]);
72             break ;
73         case 4:
74             HAL_GPIO_WritePin(GPIOA, ENM4_Pin, RESET);
75             displayMatrix(matrix_buffer[index]);
76             break ;
77         case 5:
78             HAL_GPIO_WritePin(GPIOA, ENM5_Pin, RESET);
79             displayMatrix(matrix_buffer[index]);
80             break ;
81         case 6:
82             HAL_GPIO_WritePin(GPIOA, ENM6_Pin, RESET);
83             displayMatrix(matrix_buffer[index]);
84             break ;
85         case 7:
86             HAL_GPIO_WritePin(GPIOA, ENM7_Pin, RESET);
87             displayMatrix(matrix_buffer[index]);
88             break ;
89         default :
90             HAL_GPIO_WritePin(GPIOA, LED_Pins , SET);
91             break ;

```

```

92     }
93 }
94
95 void setTimer(int index, int value){
96     timer_counter[index]=value/TIMER_TICK;
97     timer_flag[index]=0;
98 }
99
100 int isTimerExpired(int index){
101     if(timer_flag[index]==1){
102         timer_flag[index]=0;
103         return 1;
104     }
105     return 0;
106 }
107
108 void timerRun(){
109     for(int i=0;i<MAX_COUNTER;i++){
110         if(timer_counter[i]>0){
111             timer_counter[i]--;
112             if(timer_counter[i]<=0) timer_flag[i]=1;
113         }
114     }
115 }

```

Program 1.14: software_timer10.c

Trong bài này các hàm trong vòng lặp while lần lượt dùng để dịch ký tự "A" sang trái, phải so với cột ma trận, dịch lên, xuống so với hàng ma trận, cứ mỗi lần tất cả các cột đều được quét qua thì lại cập nhập giá trị mới của mỗi cột và cứ lặp lại như thế.

2 Link Github

<https://github.com/quangtrungcode/STM32LAB2.git>